

Edge cases (context limit) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below assume `client = OpenAI()` with `.env` already loaded.

Basic Version

Approach 1 — send it, catch whatever happens

```
# context_limit_practice.py
huge_input = "word " * 200_000

try:
    client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": huge_input}],
    )
except Exception as e:
    print("Error type:", type(e).__name__)
    print("Message:", str(e)[:200])
```

Expected output:

```
Error type: BadRequestError
Message: Error code: 400 - {'error': {'message': "This model's maximum context le
```

This works and shows the real error. Catching the broad `Exception` finds it, but doesn't document which specific class your code is actually expecting — see Intermediate.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — catch the specific error type

```
# context_limit_practice.py
import openai

huge_input = "word " * 200_000

try:
    client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": huge_input}],
    )
except openai.BadRequestError as e:
    print(f"Error type: {type(e).__name__}")
    print(f"Message: {str(e)[:200]}")
```

Expected output:

```
Error type: BadRequestError
Message: Error code: 400 - {'error': {'message': "This model's maximum context le
```

Difference from Basic: catching `openai.BadRequestError` specifically, instead of a bare `Exception`, documents exactly what failure this code expects and is prepared for — the same "catch the specific error you know how to handle" rule from Doc01. A bare `except Exception` here would also silently swallow a real bug elsewhere in how the request was built, not just this one expected, known failure.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — check the size before sending, using `tiktoken`

```
# context_limit_practice.py
import tiktoken

MAX_TOKENS = 120_000 # leave headroom below the model's real limit

def count_tokens(text: str, model: str = "gpt-4o-mini") -> int:
    encoding = tiktoken.encoding_for_model(model)
    return len(encoding.encode(text))

huge_input = "word " * 200_000
token_count = count_tokens(huge_input)

if token_count > MAX_TOKENS:
    print(f"Input is {token_count} tokens – too long, refusing to send.")
else:
    client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": huge_input}],
    )
```

Expected output:

```
Input is 200000 tokens – too long, refusing to send.
```

No API call happens at all — the script finishes instantly instead of waiting on a network round-trip that was always going to fail.

Approach 2 — same local check, but trim instead of just refusing

Refusing outright is the safest default, but a real chat feature usually wants to keep the conversation going by trimming the oldest content instead of stopping cold — the same trade-off

`ChatSession.send()`'s `max_turns` from the previous exercise makes for turn count, applied here directly to token count.

```
# context_limit_practice.py
import tiktoken

MAX_TOKENS = 120_000

def count_tokens(text: str, model: str = "gpt-4o-mini") -> int:
    encoding = tiktoken.encoding_for_model(model)
    return len(encoding.encode(text))

def fit_to_budget(history: list[dict], model: str = "gpt-4o-mini") -> list[dict]:
    system_message = history[0]
    rest = history[1:]

    while rest:
        full_text = system_message["content"] + "".join(m["content"] for m in rest)
        if count_tokens(full_text, model) <= MAX_TOKENS:
            break
        rest = rest[2:] # drop the oldest user/assistant pair

    return [system_message] + rest
```

Expected output, called on an oversized `history` : a shorter list, with the system message kept and the earliest user/assistant pairs dropped two at a time until the remaining text fits under `MAX_TOKENS` — the conversation continues instead of failing outright.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate only finds out about the problem *after* wasting a network round-trip on a request that was always going to fail. Approach 1 catches it locally, instantly, for free — but its answer to "the input is too big" is simply "refuse." Approach 2 builds on the same `count_tokens()` function but answers the harder design question from Hint 1: instead of refusing, it actively shrinks `history` to fit, dropping the oldest turns first, so the conversation keeps going. Neither Approach 1 nor 2 is "more correct" than the other — they're different policies for the same measured fact, and the right one depends on whether losing the oldest context silently is acceptable for your feature.

Which one should you actually ship? Real chat apps almost always do both, layered: Approach 1's local `tiktoken` check as the first, free line of defense (catching the common case before ever calling the API), Approach 2's trimming so a long conversation degrades gracefully instead of hard-failing, and Intermediate's `except openai.BadRequestError` kept as a safety net underneath both — for anything the local count got wrong, or a limit that changed since your `tiktoken` encoding was last updated.